

Week 3 • Understanding user needs

Technical Interaction Design • ITU • Autumn 2026

Built 22 September 2026 • commit 99c310c

Latest version: <https://itu-tid.github.io/lecture-notes-pdf/Week-03.pdf>

CONTENTS

The Interaction Design Process

- *Technical* Interaction Design

Tasks, as Lauesen means them

- The coffee test
- What one looks like
- Prioritising
- How this differs from a user story

The Interaction Design Process

Source: [Lectures/Interaction-Design/The-Interaction-Design-Process.md](#) — something unclear? [Open an issue](#), or send a pull request.

***"I saw the angel in the stone and I carved until I set it free."* – Michelangelo (1475 – 1564)**

An Iterative Discovery and Creation Process

ID is a creative process, and like any creative process

- has to be developed with a given set of constraints and
- has to be done in an iterative manner

Unlike other design disciplines (logo design, visual design, etc.) interaction design it is focused as much on *behavior* as on form.

The following are two popular alternative definitions of ID:

1. *ID is the practice of designing interactive digital products, environments, systems, and services.* (Alan Cooper, About Face)

2. *Designing interactive products to support the way people communicate and interact in everyday and working lives.* (Preece et. al, Interaction Design)

As you can imagine, based on these definitions, interaction design is a very broad domain. E.g. CHI – the main international conference where researchers working in human-computer interaction meet to discuss the latest findings. At one of the last such conferences I attended the topics ranged from: smart shoes, to smart pills that you could track as they traversed your digestive tract, to "5D" games, to adaptable systems for language learning (this, the least futuristic one, was my work).

How to design a successful Interaction?

Every year an incredible large number of apps is created, and only a few of them are successful. So what makes some apps more successful than others?

Think about:

- How many apps do you have installed on your phone? how many apps are out there?
- How many products did Google have? How many were eventually discontinued?
(<https://killedbygoogle.com/>)
- How many startups set out to disrupt and revolutionize the world? How many actually do this?

In Class Interactive Exercise: Think about a Shopping List app. What unique features would you add to it to make it stand out?

Every app is a point in a very large design and feature space. Both design and functionality have incredibly high dimensions. And many of the points in that space are unsuccessful. Finding the right point in that feature and behavior space is a difficult challenge. And ... it's better if you iterate.

If we think about what the successful products, they usually solve a problem. They offer us value. And most often they also do it in an efficient and pleasant way. Indeed, in a category with many competing products offering the same functionality, the ones that simpler to use, or more pleasant to use, are preferred.

Thus, we introduce the distinction between: *usefulness* and *usability* ([Laundauer]):

- **usefulness** = value delivered to the user. To have something that is useful it requires understanding humans and their context; and understanding their problems; their specific problems; and making sure that you fit in their workflow; and it is more important than usable; but it's also harder;
- **usability** means that the digital artifact is pleasant; and easy to use; not cumbersome; and not annoying to the user; and positive emotions in the user [Emotional Design, D. Norman]; some people consider usability to be an [excavator of usefulness](#).

Note that sometimes, usability is just a nice to have thing; and other times it's a matter of life and death. The story of the car gear [shifter in which the state was not visible](#). Or the low usability of the plane controls that lead to the [death of John Denver](#).

It seems that delivering something useful and usable are both important. But how to do that?

- For Usefulness one must understand your users and their needs. Think about movies - how do they select their audience?
- For Usability one must be aware of usability principles but also know how to evaluate with actual users

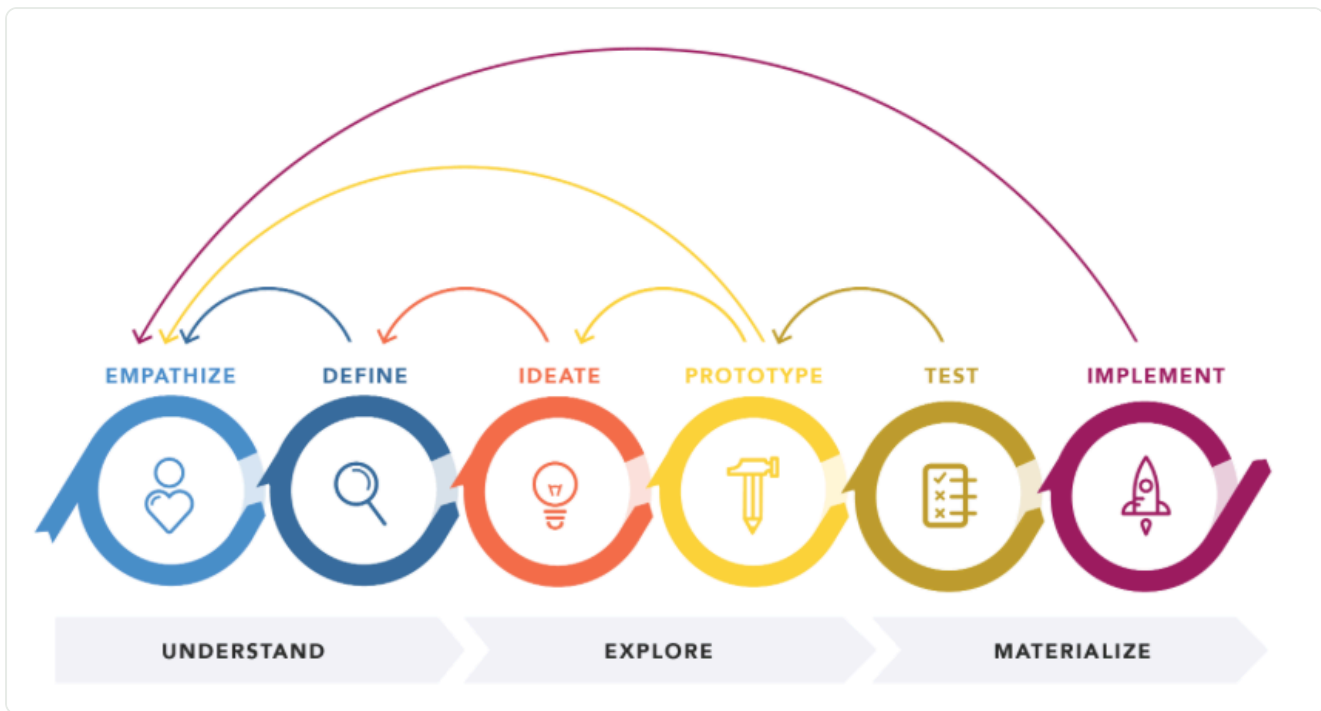
One good place to start from is knowing who your users are. Designing for the average user means frustrating most of the users. [The Flaw of the Averages](#)

Specific Processes & Approaches

Given the importance of designing interactions in the world we live in and our fondness of having a "process" for everything we do (SCRUM anybody?) you can imagine why many people would try to provide recipes for how to do Interaction Design. And indeed, there are many processes for ID. The most popular one, and the one from one of the most popular company out there are:

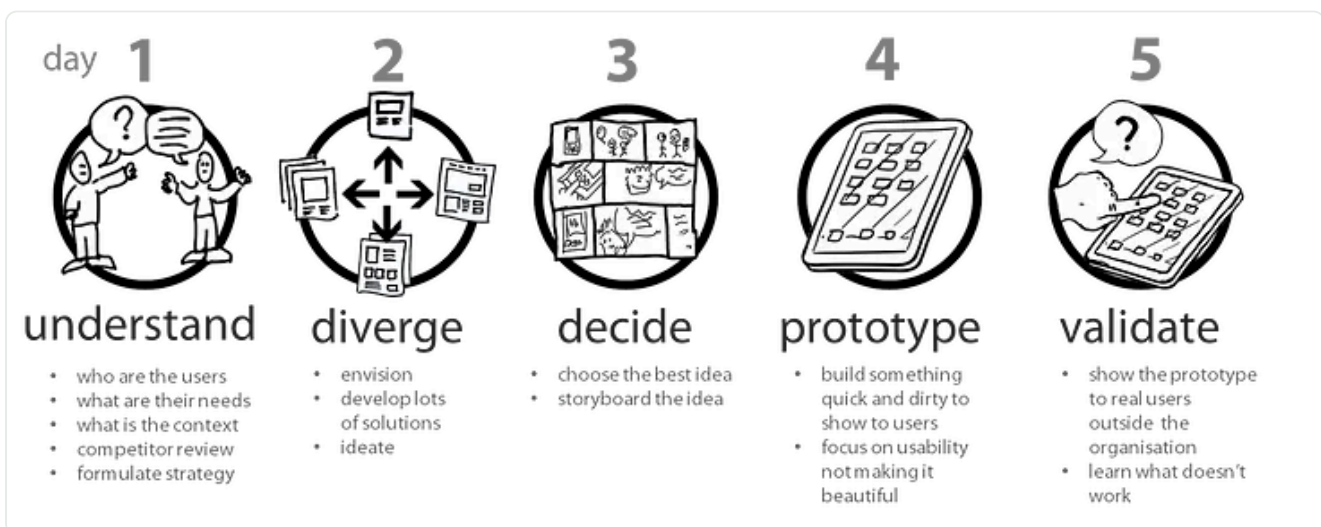
Design Thinking

An ideology that asserts that a hands-on, user-centric approach to problem solving can lead to innovation [DT]. Process comprises 6 distinct phases.



Google Design Sprints

A process that starts from ideation to prototyping and evaluation but explicitly without implementation [GDS].



- [1min summary video](#) - [10 min overview video](#)

All of the processes are iterative. And, most of them, including the ones above, have several high-level activities in common:

- understanding - collecting data about the users and the problem that needs to be solved
- generating alternatives - getting creative in discovering solutions to the problems; brainstorming
- prototyping - trying out stuff as early and as cheaply as possible
- evaluating - getting feedback from representative users; observing them

Different approaches put different emphasis on the different kinds of phases but all the phases are important. In every phase, there are different kinds of activities.

Finally, most processes will make a distinction between phases in which one searches for solution and phases in which these solutions are reduced and evaluated. This is the main idea behind the **Double Diamond Model** - a process model created by Design Council, a British organization, in 2005. It highlights four main stages across two adjacent diamonds:

- **Discover.** The first diamond helps people understand, rather than simply assume, what the problem is
- **Develop.** The second diamond encourages people to give different answers to the clearly defined problem

Finally, although not as process-driven, there are other approaches can also work sometimes:

- **Wizard Design** - this is what Steve Jobs used to do. He had a very strong intuition of what people needed and created it for them. Henry Ford: "if you ask people, they will want a faster horse".
- **Design for yourself** - this is what the authors of Vi, LaTeX, etc. set out to do. Solving a problem that they had. And then, it turned out, that other people also had that problem. Is this a more or less risky approach than Design-Driven Innovation?

***Technical* Interaction Design**

This course is about *technical* ID. This means that we focus much more on the technical aspects of creating interactions. In particular we will discuss several technologies that you will likely need for the foreseeable future if you want to create any interactive application:

- Javascript - We will go together in the first weeks through the important parts just to make sure that everybody is onboard when we hit the React "wall".
- CSS - We will cover some of the essential layout-related topics together; I expect you to have a basic understanding of it; however you'll see - it's more stuffy that it looks
- React - We will cover everything from beginner to advanced topics
- Git - A collaboration tool that you can not live without

These technologies are useful for almost any web development project and also for many mobile applications at the moment. E.g., React Native is very similar to React and allows you to build cross-platform applications.

References

[Laundauer] - The Trouble with Computers: Usefulness, Usability, and Productivity, Laundauer. [pp.141 – 142.](#)

[DT] [Design Thinking 101](#), Sara Gibbons.

[GDS] [How Google Design Sprint Works](#) Thaisa Fernandes, 5min read.

Something unclear in this section? Open an issue.

Tasks, as Lauesen means them

Source: [Lectures/Interaction-Design/Tasks-Lauesen.md](#) — something unclear? [Open an issue](#), or send a pull request.

Interviews give you what people said. Analysing them gives you what keeps coming up. Neither of those is something an app can do, so something has to turn a finding into something you could build.

There are several ways to write that down: user stories, the task table from the user-research lecture, and this one. This note is Søren Lauesen's version, and the word **task** means something specific in it.

It is the strictest of the three, and useful for one reason: it asks what people were doing *before* your app existed, which is how it can tell you the answer was not the thing you assumed. Use it if it helps you. Nothing requires it — what the hand-in asks for, and in which format, is on learnIT.

The coffee test

A task is something that, once you have finished it, you can go and have a coffee.

That is the whole definition, and it does more work than it looks. It rules out almost everything people write down when asked for requirements.

Tasks:

- Do this week's shopping for the flat
- Get the kids' packing list ready the night before a trip
- Work out what still needs doing before the guests arrive

Not tasks:

what it is	why it fails	example
a click	you cannot stop there	"tap the checkbox"
a screen	screens are your answer, not their need	"the settings page"
a feature	that is a solution in disguise	"push notifications"
a whole life goal	too big to finish, ever	"be more organised"

The middle two are the ones you will get wrong. A task describes what someone is trying to accomplish **before they have ever seen your app** — so if you could not have written it down after the interview, without knowing what you were going to build, it is not a task yet.

Something unclear in this section? [Open an issue](#).

What one looks like

Four fields. Keep it short; a task that runs to a page is two tasks.

Name — a verb phrase, in the user's words, not yours.

Trigger — what starts it. Usually a moment in the day or an event, not a decision to open an app. This is the field people skip and it is the one that tells you when your app has to be reachable, and on what.

Sub-steps — what happens, in order, as it happens now. Still no app: you are recording reality, not designing.

Variants — what happens when it does not go straightforwardly. This is where the design work actually lives. Most apps handle the happy path and fall apart here.

Worked example

Name. Do this week's shopping for the flat.

Trigger. Someone notices the milk has run out, or it is Saturday morning.

Sub-steps.

1. Work out what is missing — from memory, and by looking in the fridge
2. Ask the others whether they need anything
3. Write it down somewhere, on paper or in a phone note
4. Go to the shop
5. Cross things off while walking around
6. Notice, at the till, the thing that was forgotten

Variants.

- Someone else has already bought half of it
- A flatmate remembers something after you have left the flat
- The shop does not have it, and you need to know whether a substitute is acceptable
- Two people go shopping on the same day without knowing

Look at what falls out of that. Step 2 and the first variant are the entire argument for sharing a list rather than owning one. Step 5 says the app has to work one-handed, in a shop, on a phone. The second variant says changes have to arrive on someone else's phone while they are out. None of that came from a feature list — it came from writing down what already happens.

That is why this is worth the afternoon it takes.

Something unclear in this section? [Open an issue.](#)

Prioritising

You will end up with more tasks than you can build. Rank them, and say why. Two questions are usually enough:

- **How often does it happen?** Daily beats monthly.
- **How badly does it go now?** A task people already manage fine is a poor place to spend your one twist.

Frequent and painful goes first. Rare and painful is where the interesting design problems live, but it is not where you start. Frequent and fine is a trap: it looks like a big win and changes nothing.

That is Lauesen's criterion, and it ranks *problems*. The hand-in asks you to rank by something else — see learnIT. They are not the same question.

Something unclear in this section? [Open an issue.](#)

How this differs from a user story

You have probably met *user stories* — “As a flatmate, I want to share a list, so that we do not buy milk twice.” They are not wrong, and industry uses them everywhere. The difference worth knowing is that a user story has the solution built into it. The sentence above already decided there is a shared list. A task does not decide anything — it records what someone is trying to do, and leaves you free to find out that the answer was not a shared list at all.

Something unclear in this section? [Open an issue.](#)